EL KATTOUFI Youssef

Année 2025 – 2026

Serrure électronique



Etablissement scolaire : Polytech Sorbonne

Formation : Electronique et Informatique

Matières : Electronique numérique

Enseignant : Yan DOUZE

Sommaire

Remerciements 4

Résumé 5

I. Présentation du projet 6

II. Conception et Architecture Fonctionnelle 7

A. Détecteur de front 7

B. Bascule D active sur front montant 9

C. Registre à changement parallèle (4bits) 10

D. Stockage du code 11

E. Affichage 7 segments 13

F. Comparateur 4bits 14

G. Comparateur 16 bits 15

III. Assemblage : Serrure électronique complète 17

A. Fonctionnement top module 17

B. Fonctionnement test bench 17

IV. Quartus prime 18

Sources 19

Tables des figures 21

Remerciements

Je tiens tout d'abord à remercier chaleureusement Monsieur François Pecheux, Monsieur Yan Douze ainsi que Monsieur David Martin, qui nous ont encadrés tout au long du semestre 5. Leur accompagnement, leurs conseils et leurs enseignements nous ont permis de mener à bien ce projet, tout en nous apportant le socle de connaissances nécessaires en électronique numérique.

Résumé

Au sein de la formation d'ingénieur en Électronique et Informatique à Polytech Sorbonne, nous avons suivi l'enseignement d'électronique numérique, qui nous a permis d'acquérir des connaissances fondamentales dans ce domaine. Cette matière nous a notamment introduits au fonctionnement des FPGA, des ASIC, ainsi qu'à l'utilisation du langage de description matérielle VHDL.

Cet enseignement m'a permis de prendre conscience de la complexité des circuits en électronique numérique, tant du point de vue de leur conception que de leur mise en œuvre. Il met en évidence l'importance de la modélisation et de l'anticipation du comportement d'un système, afin de prévoir son fonctionnement avant toute réalisation matérielle.

I. Présentation du projet

Ce mini-projet a pour objectif la conception et la simulation d'une serrure électronique en VHDL sur la carte DE10-Lite (voir figure ci-dessous). Le système permet à un utilisateur de saisir un code secret à quatre chiffres à l'aide des interrupteurs, de valider chaque chiffre via un bouton poussoir, puis d'afficher dynamiquement la saisie sur des afficheurs 7 segments.

À chaque validation, le chiffre entré est affiché tandis que les précédents sont décalés, jusqu'à la saisie complète du code. Une fois les quatre chiffres enregistrés, le système compare le code utilisateur au code secret prédéfini (ici 1234) et déclenche l'ouverture de la serrure, matérialisée par l'allumage d'une LED, en cas de correspondance.

Ce type de dispositif est notamment utilisé dans des applications telles que les coffres-forts électroniques d'hôtels. Ce projet constitue une introduction pratique à l'électronique numérique et vise à mettre en œuvre la conception de blocs combinatoires (comparateur, affichage), de blocs séquentiels synchrones (détecteur de front, mémoire à décalage), ainsi que la structuration hiérarchique d'un système numérique complet, en validant chaque module par simulation avant l'intégration finale.

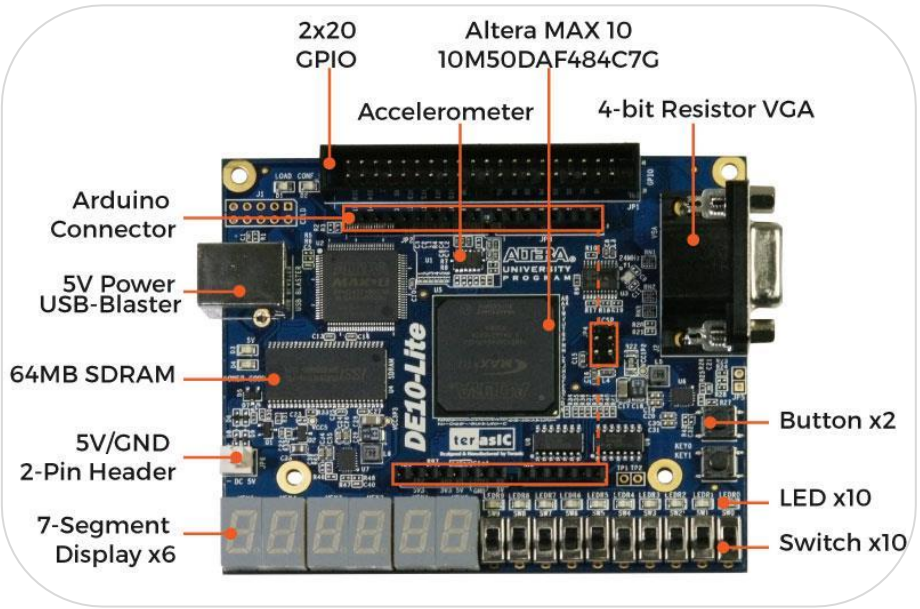


Figure 1 : Carte DE10-Lite

II. Conception et Architecture Fonctionnelle

Dans ce compte-rendu, nous présentons la démarche complète de conception de la serrure électronique, en détaillant la réflexion, les choix de conception, les schémas, le code VHDL et les simulations pour chaque module, afin de permettre de comprendre étape par étape le fonctionnement du système :

- Détecteur de front ;
- Bascule D activée sur front montant ;
- Registre à chargement parallèle ;
- Stockage du code ;
- Affichage sur les 7 segments ;
- Comparateur 4 bits ;
- Comparateur 16 bits ;

La méthode de projet adoptée a consisté à organiser clairement les fichiers afin d'éviter toute confusion. Pour cela, j'ai créé deux dossiers : un dossier Modelsim, regroupant toute la partie relative à la modélisation et à la simulation du code VHDL, et un dossier Quartus Prime, permettant l'upload du code fonctionnel sur la carte DE10-Lite ainsi que sa synthèse sur le FPGA. Par ailleurs, afin de conserver une trace complète de l'évolution du projet, j'ai créé un dépôt GitHub, permettant de suivre l'historique des modifications et de gérer efficacement les différentes versions du projet.

Pour chacune de ces parties, il existe un fichier simu.do permettant d'automatiser les tests benchs.

A. Détecteur de front

Dans la partie 4.1 du TP, il nous a été demandé de concevoir un détecteur de front montant. Dans un premier temps, cette implémentation fonctionnait correctement en simulation sous ModelSim. Cependant, lors des tests pratiques sur la carte DE10-Lite, un comportement inattendu a été observé : le système ne validait pas la saisie lors de l'appui sur le bouton poussoir, mais uniquement lors de son relâchement.

Cette observation s'explique par le fait que le bouton poussoir de la carte est actif à l'état bas (pull-down). Ainsi, le signal passe de '1' à '0' lors de l'appui, ce qui correspond à un front descendant. Afin d'obtenir un comportement cohérent entre la simulation et le fonctionnement réel sur la carte, le code du détecteur de front ainsi que le banc de test ont été modifiés pour détecter un front descendant plutôt qu'un front montant.

Sur la figure suivante, plusieurs éléments peuvent être observés :

- Détection du front descendant : le signal Detectar passe à '1' uniquement lorsque KEY0 effectue une transition de '1' vers '0'.
- Comportement séquentiel : Detectar ne change pas d'état instantanément lors de la transition de KEY0, mais uniquement au front montant de l'horloge (clk) suivant, ce qui correspond au comportement attendu d'un circuit synchrone utilisant rising_edge(clk).
- Durée de l'impulsion : l'impulsion sur Detectar dure exactement une période d'horloge (10 ps), puis revient à '0' car la condition de détection n'est plus satisfaite au cycle suivant.
- Initialisation (Reset) : au début de la simulation, lorsque rst_n est à '0', tous les signaux sont correctement initialisés, évitant ainsi l'apparition de valeurs indéfinies.
- Sur la simulation présentée :
- Premier front descendant : KEY0 passe à '0' vers 35 ps ; au front d'horloge suivant, à 45 ps, le signal Detectar passe à '1'.
- Second front descendant : KEY0 passe à '0' vers 75 ps ; au front d'horloge de 85 ps, Detectar est de nouveau activé.

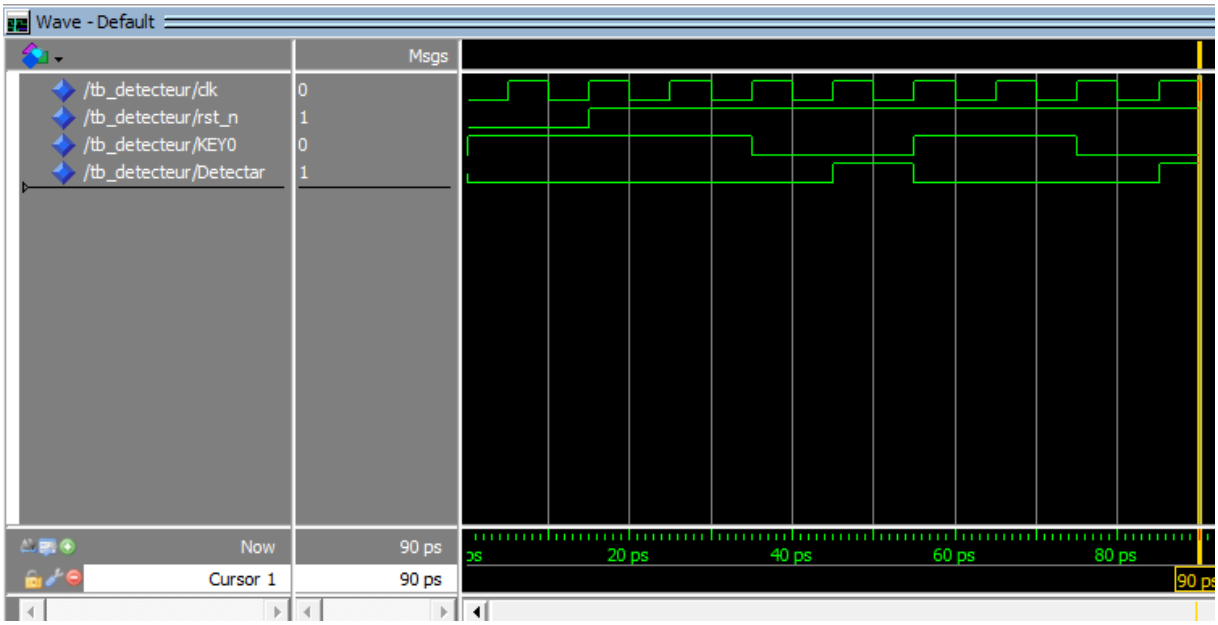


Figure 2 : Simulation détecteur de front

B. Bascule D active sur front montant

Dans la partie 4.2, il nous a été demandé de décrire et d'implémenter le fonctionnement d'une bascule D déclenchée sur le front montant de l'horloge.

Une bascule D est un circuit logique séquentiel capable de mémoriser un bit (0 ou 1). Contrairement aux circuits combinatoires, son état de sortie dépend non seulement des entrées actuelles, mais aussi de son état précédent. La bascule change d'état en fonction d'un signal d'horloge, ce qui permet de synchroniser les opérations dans un système numérique. Selon son type, elle définit des règles précises pour le stockage, la mise à jour ou la conservation de l'information, et constitue un élément de base des registres, compteurs et mémoires.

L'observation du chronogramme issu de la simulation permet de valider le bon fonctionnement de la bascule D. Durant la phase de reset, comprise entre 0 et 15 ps, le signal rst_n est maintenu à '0', ce qui force la sortie Q à l'état bas indépendamment de l'horloge et des variations du signal D. Lorsque rst_n repasse à '1', la bascule reprend un fonctionnement normal. Entre 15 et 25 ps, bien que D passe à '1', la sortie Q reste inchangée car le signal enable est à '0'. Par la suite, lorsque enable devient actif, la valeur de D est prise en compte au front montant de l'horloge suivant, ce qui provoque le passage de Q à '1'. Une nouvelle variation de D à '0' est ensuite correctement répercutée sur la sortie au front d'horloge suivant, tant que enable reste actif. Enfin, lorsque enable repasse à '0', la sortie Q conserve sa valeur, confirmant le bon comportement de mémorisation de la bascule.

Le code VHDL développé respecte les règles de conception d'une bascule D synchrone avec remise à zéro asynchrone. La priorité donnée au signal rst_n en dehors de la condition de détection du front montant de l'horloge garantit une remise à zéro immédiate et fiable. La gestion de l'entrée enable à l'intérieur du rising_edge(clk) permet d'implémenter précisément le mécanisme de mémorisation conditionnelle attendu. Enfin, l'utilisation d'un signal intermédiaire pour stocker l'état interne de la bascule améliore la lisibilité du code et constitue une bonne pratique en conception VHDL, tout en facilitant la compréhension et la maintenance du module.

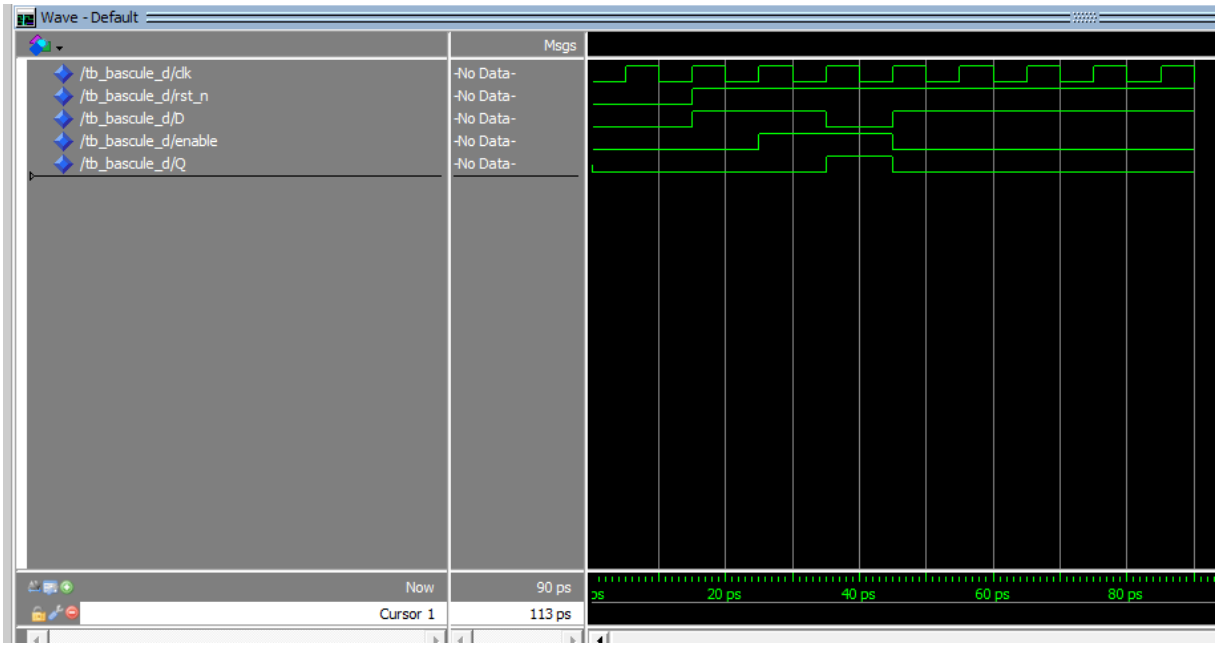


Figure 3 : Simulation bascule D

C. *Registre à changement parallèle (4bits)*

L'entité *registre_changement_parallele* définit les entrées et sorties du module : une horloge (*clk*), un signal de remise à zéro asynchrone actif à l'état bas (*rst_n*), un signal d'activation (*enable*), un bus de données d'entrée sur 4 bits (*D*) et un bus de sortie sur 4 bits (*Q*). Ce registre permet de mémoriser simultanément les quatre bits du bus *D* lorsque l'activation est autorisée et qu'un front montant de l'horloge se produit.

Dans l'architecture *Structural*, nous avons choisi d'instancier manuellement quatre bascules D, correspondant chacune à un bit du registre. Chaque instance de la bascule *bascule_D* est connectée au même signal d'horloge, au même reset asynchrone et au même signal *enable*, garantissant ainsi un fonctionnement synchrone de l'ensemble du registre. Chaque bascule mémorise un bit spécifique du bus *D* (de *D(0)* à *D(3)*) et fournit le bit correspondant en sortie sur cette sortie *Q*.

Afin de vérifier le bon fonctionnement de notre registre, nous avons développé un banc de test (tb_registre) permettant de simuler le comportement du composant dans des conditions réelles. L'horloge est générée de manière autonome avec une période de 10 ps via l'instruction clk <= not clk after 5 ps;. Le registre précédemment conçu est instancié dans le banc de test comme unité à stimuler (Unit Under Test). Le processus de simulation suit une séquence chronologique pour valider le code : l'initialisation est réalisée en activant le reset (rst_n <= '0') afin de s'assurer que le registre démarre dans un état connu, dans notre cas, normalement c'est « 0000 ». Ensuite, le

reset est levé, la donnée « 1010 » est appliquée sur l'entrée D et le signal enable est activé ; au front montant de l'horloge suivant, la sortie Q doit recopier cette valeur. Enfin, lorsque enable est désactivé et que l'entrée D est modifiée à « 1111 », le test permet de vérifier que la sortie Q conserve l'ancienne valeur « 1010 », démontrant que le verrouillage des données fonctionne correctement malgré les changements sur le bus d'entrée.

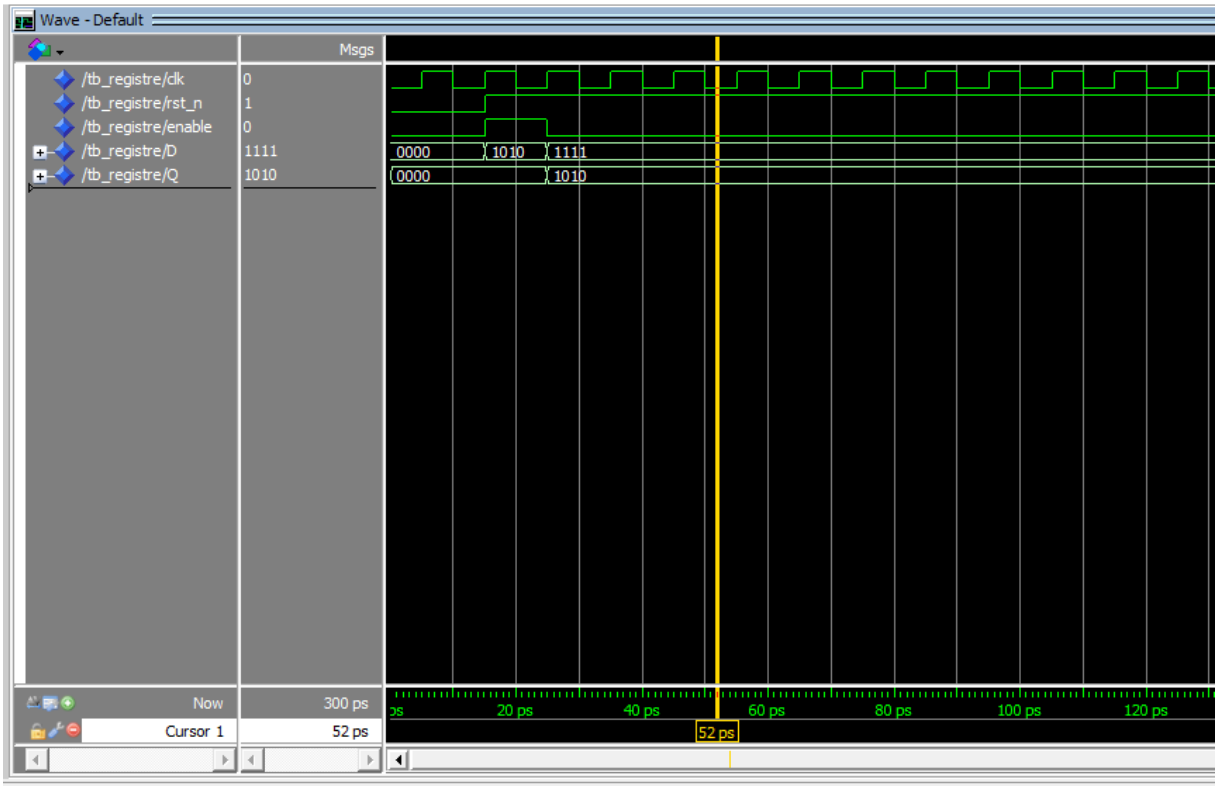


Figure 4 : Simulation registre à changement parallèle

D. *Stockage du code*

Pour réaliser une structure de stockage sur 16 bits en va chaîné quatre registres de 4 bits. Chaque registre mémorise un chiffre saisi successivement, ce qui conduit à la constitution progressive d'un mot de 16 bits représentant l'ensemble du code utilisateur.

Le décalage des données est assuré de manière synchrone. L'ensemble des registres partage le même signal d'horloge ainsi que le même signal d'activation, issu du détecteur de front. Ainsi, à chaque front montant de l'horloge, le premier registre charge la nouvelle valeur présente sur le bus intermédiaire (*Busint*), tandis que les registres suivants récupèrent simultanément la valeur précédemment stockée dans le registre qui les précède. Cela permet un décalage simultané des données dans toute la chaîne de registres.

L'organisation des bits au sein du bus final *Buscode* respecte correctement l'ordre des poids. La concaténation des sorties *q4*, *q3*, *q2* et *q1* place le dernier chiffre saisi (*q1*) sur les bits de poids

faible (bits 0 à 3), tandis que le premier chiffre saisi (*q4*) est positionné sur les bits de poids fort (bits 12 à 15).

Enfin, le cheminement des données est défini : le bus intermédiaire *Busint* alimente l'entrée du premier registre, la sortie *q1* de ce registre est connectée à l'entrée du second, puis *q2* alimente le troisième registre et *q3* le quatrième. Les quatre sorties sont ensuite regroupées pour former un bus de 16 bits unique, *Buscode*, représentant l'état complet du code mémorisé par le système.

Voici le schéma du circuit :

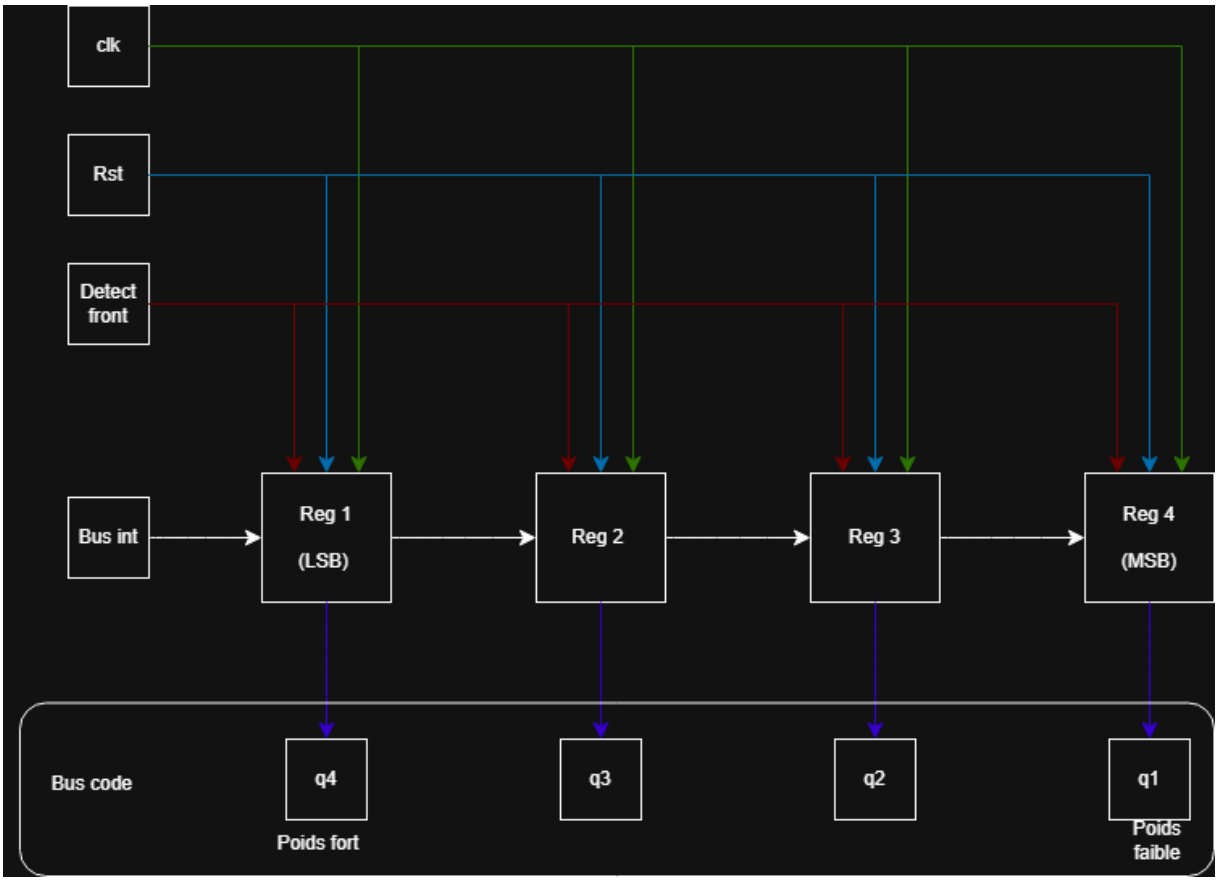


Figure 5 : Logigramme du stockage code

Aucun module code VHDL propre, ni simulation n'a été réalisé, cette fonction a été intégrée directement au sein du module principal (top module), en s'appuyant sur les registres précédemment conçus et validés. Le stockage du code a fonctionné correctement dès la première implémentation, sans nécessiter de modifications supplémentaires. Quatre registres à chargement parallèle sont instanciés en cascade afin de réaliser le décalage successif des données : le premier registre reçoit la valeur d'entrée, tandis que chaque registre suivant récupère la sortie du précédent à chaque activation du signal issu du détecteur de front. Les quatre sorties sont ensuite concaténées pour former un bus interne de 16 bits représentant le code complet

mémorisé. Ce bus est enfin utilisé comme entrée du comparateur 16 bits, permettant la validation du code saisi par l'utilisateur.

```
-- Instanciation des 4 registres pour le décalage
-- REG1 reçoit l'entrée en premier
REG1 : entity work.registre_changement_parallele
    port map (clk => clk, rst_n => rst_n, enable => detect_front, D => Busint, Q => q1);

-- REG2 reçoit ce qui sort de REG1
REG2 : entity work.registre_changement_parallele
    port map (clk => clk, rst_n => rst_n, enable => detect_front, D => q1, Q => q2);

-- REG3 reçoit ce qui sort de REG2
REG3 : entity work.registre_changement_parallele
    port map (clk => clk, rst_n => rst_n, enable => detect_front, D => q2, Q => q3);

-- REG4 reçoit ce qui sort de REG3
REG4 : entity work.registre_changement_parallele
    port map (clk => clk, rst_n => rst_n, enable => detect_front, D => q3, Q => q4);

-- On regroupe tout pour la sortie 16 bits    youssef [2 weeks ago] • Modelsim foncitonne +
buscode_interne <= q4 & q3 & q2 & q1;
```

Figure 6 : Code extrait top module permettant le stockage du code

E. Affichage 7 segments

Le code du décodeur 7 segments utilise l'instruction « with...select ». Grâce à cette intruction on convertit un signal d'entrée sur 4 bits en une combinaison de 7 bits, où chaque bit correspond à un segment physique (de a à g). En regardant la CAO électronique de la carte DE10-Lite, la logique celle ou active à l'état bas ou le 0, la masse du circuit allume le segment visé. La table de correspondance traduit les valeurs binaires de 0000 à 1111 en caractères alphanumériques allant de 0 à 9 et de A à F.

Le test bench et la simulation associée valident cette table de conversion en appliquant des représentation sur l'entrée tb_entree. Sur le chronogramme obtenu, on observe la sortie des segments à chaque changement de valeur d'entrée.

Concernant l'état initial, l'afficheur segment est par défaut éteint.

Ci-dessous le schéma du composant :

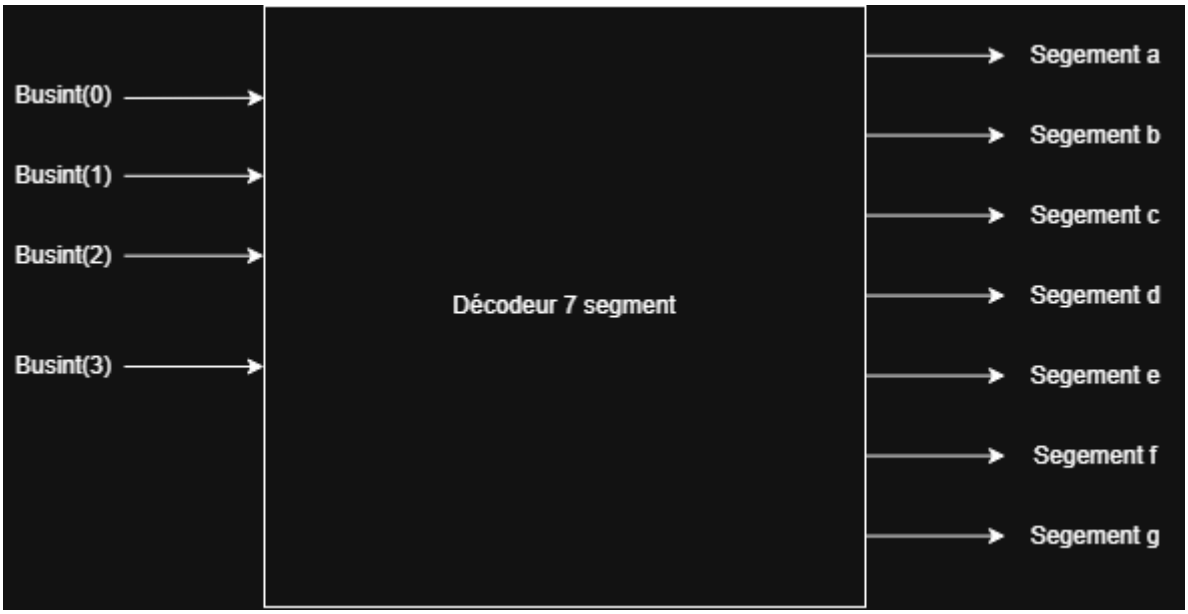


Figure 7 : Schéma de ce composant

Ci-joint la simulation :



Figure 8 : Simulation décodeur 7 segment

F. *Comparateur 4bits*

Dans ce comparateur 4 bits, il n'est pas nécessaire d'utiliser un signal de reset pour remettre la sortie à zéro. En effet, le module est entièrement combinatoire : la sortie *CompOK* dépend uniquement des valeurs instantanées des entrées *Datautil* et *Dataamd*. Dès que ces deux vecteurs sont différents, la sortie prend automatiquement la valeur `'0'`, sans avoir besoin de

mémoriser un état précédent. Un signal de reset est surtout utile dans les circuits séquentiels (bascules, registres, compteurs), ce qui n'est pas le cas ici.

Le module VHDL est décrit à l'aide d'une entité comparateur_4bits qui définit deux entrées sur 4 bits (*Datautil* et *Dataamd*) ainsi qu'une sortie (*CompOK)*. L'architecture de type *Behavioral* implémente une affectation conditionnelle simple : la sortie *CompOK* vaut '1' lorsque les deux entrées sont strictement égales, et '0' dans tous les autres cas. Cette écriture est claire, synthétisable et bien adaptée à un comparateur combinatoire.

Dans cette simulation, deux signaux internes *s_data1* et *s_data2* sur 4 bits représentent respectivement les entrées *Datautil* et *Dataamd*, tandis que le signal *s_ok* correspond à la sortie *CompOK*. Le composant comparateur_4bits est instancié sous le nom UUT (Unit Under Test) et connecté à ces signaux.

Le processus de simulation applique ensuite plusieurs tests successifs. Dans un premier temps, *s_data1* vaut "1010" et *s_data2* vaut "0000" : les valeurs étant différentes, la sortie *s_ok* reste à '0'. Dans un second test, les deux entrées sont mises à "1100", ce qui provoque le passage de *s_ok* à '1', confirmant que le comparateur détecte correctement l'égalité. Enfin, un troisième test applique des valeurs différentes ("1100" et "1111"), et la sortie repasse à '0'. Ces résultats observés en simulation valident le bon fonctionnement du comparateur 4 bits conformément au cahier des charges.

Ci-joint la simulation réussi :



Figure 9 : Simulation comparateur 4 bits

G. Comparateur 16 bits

Le comparateur 16 bits est décrit à l'aide d'une architecture de type *Structural*, ce qui permet de réutiliser directement le composant *comparateur_4bits* déjà conçu. L'entrée *Buscode* est un vecteur de 16 bits découpé en quatre groupes de 4 bits, chacun correspondant à un chiffre du

code saisi. Quatre instances du comparateur 4 bits sont alors utilisées : chaque instance compare une portion de *Buscode* avec une valeur de référence fixe (0100 pour 4, 0011 pour 3, 0010 pour 2 et 0001 pour 1). Les sorties de ces comparateurs sont stockées dans les signaux internes *ok0* à *ok3*.

Le fonctionnement du comparateur 16 bits repose sur une logique combinatoire. Chaque comparateur 4 bits vérifie si la partie du bus qui lui est associée correspond à la valeur attendue. La sortie finale *ouverture* est obtenue par une porte logique *ET* entre les signaux *ok0, ok1, ok2* et *ok3*. Ainsi, *ouverture* prend la valeur `'1'` uniquement si les quatre comparaisons sont correctes simultanément, c'est-à-dire si le code complet saisi correspond exactement au code secret attendu. Dans tous les autres cas, la sortie reste à `'0'`.

La simulation est réalisée à l'aide du banc de test *tb_comparateur_16bits*. Celui-ci génère différentes valeurs sur le signal *tb_bus*, connecté à l'entrée *Buscode* du comparateur 16 bits. Un premier test applique un code incorrect (`"1000010000100001"`, correspondant à 8421), pour lequel au moins un des comparateurs 4 bits détecte une différence. La sortie *tb_ouverture* reste alors à `'0'`. Dans un second test, le code correct (`"0001001000110100"`, correspondant à 1234) est appliqué : les quatre comparateurs 4 bits valident chacun leur portion du bus, ce qui entraîne le passage de *tb_ouverture* à `'1'`. Les résultats observés en simulation confirment donc le bon fonctionnement du comparateur 16 bits et la cohérence avec le cahier des charges.

Ci-joint simulation :

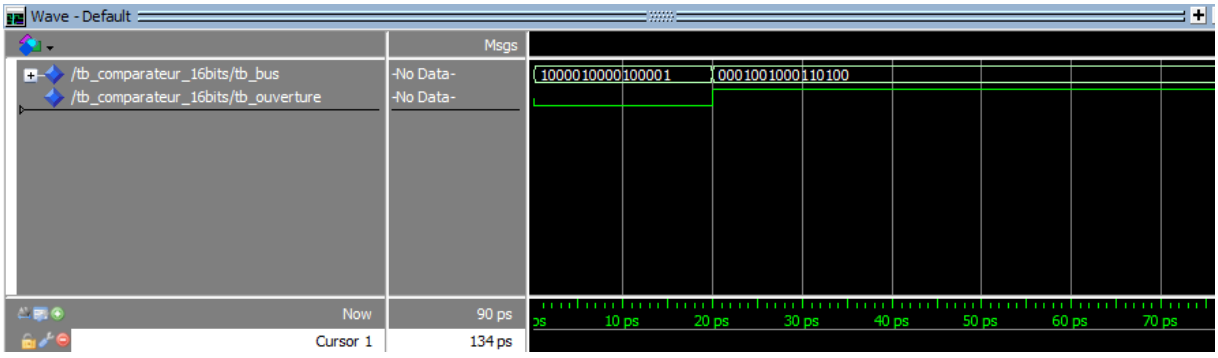


Figure 10 : Simulation comparateur 16 bits

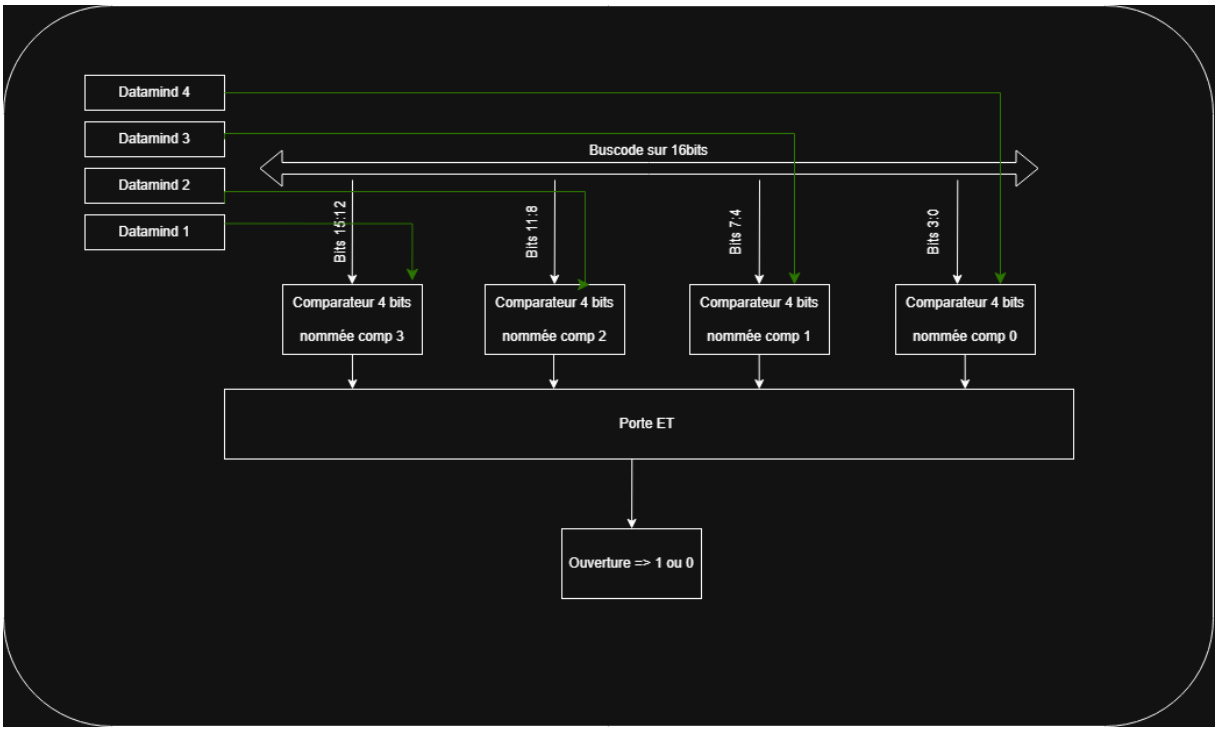


Figure 11 : Schéma Comparateur 16 bits

III. Assemblage : Serrure électronique complète

A. Fonctionnement top module

Le module principal top_module représente le fonctionnement global de la serrure numérique. Il est structuré de manière hiérarchique et relie plusieurs sous-blocs fonctionnels. Le signal KEY0 est d'abord traité par un détecteur de front, qui génère une impulsion (detect_front) à chaque appui sur le bouton. Cette impulsion permet de valider la valeur présente sur Busint, qui correspond au chiffre saisi par l'utilisateur. La valeur est ensuite mémorisée dans une chaîne de quatre registres fonctionnant en décalage : à chaque validation, le nouveau chiffre est stocké dans le premier registre et les précédents sont décalés vers les suivants. Les quatre registres forment ainsi un bus interne de 16 bits représentant le code complet saisi. Ce bus est comparé à un code de référence par le comparateur 16 bits, et si les quatre chiffres sont corrects, la sortie Ouverture est mise à '1'. En parallèle, chaque registre est connecté à un décodeur 7 segments afin d'afficher les chiffres saisis sur les afficheurs HEX0 à HEX3.

B. Fonctionnement test bench

Le banc de test top_module_tb permet de vérifier le comportement du système sans matériel réel. Il génère d'abord une horloge périodique et pilote le signal de reset afin d'initialiser correctement le système. Ensuite, le testbench simule la saisie d'un code en appliquant successivement différentes valeurs sur Busint et en générant des impulsions sur KEY0, ce qui

reproduit l'action d'un utilisateur appuyant sur un bouton de validation. Deux scénarios sont testés : un premier avec un code correct (1-2-3-4), pour lequel la sortie Ouverture doit passer à '1', et un second avec un code incorrect, pour lequel la sortie reste à '0'. Les délais introduits entre chaque action permettent d'observer clairement l'évolution des signaux en simulation et de valider le fonctionnement global de la serrure numérique. Il a été découvert sur internet qu'il existait la possibilité de nommée ces process, c'est ce que j'ai fait pour des raisons de lecture de code.

Il y a également un fichier simu.do permettant d'automatiser les tests.

IV. Quartus prime

Pour la partie réalisée avec Quartus Prime, j'ai choisi de ne pas nommer le projet *serrure_electronique* mais top_module. Ce choix a été fait afin de rester cohérent avec la base de travail déjà effectué sous ModelSim. L'ensemble des simulations fonctionnait correctement, et par précaution, j'ai préféré ne pas modifier le nom du fichier ni celui de l'entité VHDL, afin d'éviter tout risque de dysfonctionnement. Le projet Quartus porte donc le nom *top_module* et est organisé dans un dossier dédié nommé *Quartus*.

Dans cette partie du projet, je me suis appuyé sur la documentation et la CAO électronique de la carte DE10-Lite afin d'identifier le raccordement des broches (*pins*). Une difficulté rencontrée concernait l'apparition de broches non souhaitées dans Quartus Prime, le *buscode*. Dans un premier temps, j'ai laissé Quartus affecter automatiquement les broches, en considérant que l'outil pouvait adapter les connexions en fonction de la carte électroniqeu. Cependant, cette configuration automatique ne donnait pas le résultat attendu, voir même ne fonctionnait pas.

J'ai alors choisi de revenir sur mes fichiers VHDL dans l'environnement ModelSim et VScode afin d'analyser, corriger et valider le fonctionnement du code par simulation. Une fois le comportement confirmé à priori, j'ai copié les fichiers VHDL validés dans le dossier du projet Quartus. Après cette mise à jour, les broches problématiques sont apparues comme *Unknown*, ce qui indique normalement qu'elles sont désormais gérées en interne et qu'il n'est plus nécessaire de les affecter.

All Pins

Named: * Edit: Filter: Pins: all

Node Name	Direction	Location	I/O Bank	VREF Group	Fitter Location	I/O Standard	Reserved	Current Strength	Slew Rate	Differential Pair	Strict Preservation
Busint[3]	Input	PIN_C12	7	B7_N0	PIN_C12	3.3-V LVTTL		8mA (default)			
Busint[2]	Input	PIN_D12	7	B7_N0	PIN_D12	3.3-V LVTTL		8mA (default)			
Busint[1]	Input	PIN_C11	7	B7_N0	PIN_C11	3.3-V LVTTL		8mA (default)			
Busint[0]	Input	PIN_C10	7	B7_N0	PIN_C10	3.3-V LVTTL		8mA (default)			
clk	Input	PIN_P11	3	B3_N0	PIN_P11	3.3-V LVTTL		8mA (default)			
HEX0[6]	Output	PIN_C17	7	B7_N0	PIN_C17	3.3-V LVTTL		8mA (default)	2 (default)		
HEX0[5]	Output	PIN_D17	7	B7_N0	PIN_D17	3.3-V LVTTL		8mA (default)	2 (default)		
HEX0[4]	Output	PIN_E16	7	B7_N0	PIN_E16	3.3-V LVTTL		8mA (default)	2 (default)		
HEX0[3]	Output	PIN_C16	7	B7_N0	PIN_C16	3.3-V LVTTL		8mA (default)	2 (default)		
HEX0[2]	Output	PIN_C15	7	B7_N0	PIN_C15	3.3-V LVTTL		8mA (default)	2 (default)		
HEX0[1]	Output	PIN_E15	7	B7_N0	PIN_E15	3.3-V LVTTL		8mA (default)	2 (default)		
HEX0[0]	Output	PIN_C14	7	B7_N0	PIN_C14	3.3-V LVTTL		8mA (default)	2 (default)		
HEX1[6]	Output	PIN_B17	7	B7_N0	PIN_B17	3.3-V LVTTL		8mA (default)	2 (default)		
HEX1[5]	Output	PIN_A18	7	B7_N0	PIN_A18	3.3-V LVTTL		8mA (default)	2 (default)		
HEX1[4]	Output	PIN_A17	7	B7_N0	PIN_A17	3.3-V LVTTL		8mA (default)	2 (default)		
HEX1[3]	Output	PIN_B16	7	B7_N0	PIN_B16	3.3-V LVTTL		8mA (default)	2 (default)		
HEX1[2]	Output	PIN_E18	6	B6_N0	PIN_E18	3.3-V LVTTL		8mA (default)	2 (default)		
HEX1[1]	Output	PIN_D18	6	B6_N0	PIN_D18	3.3-V LVTTL		8mA (default)	2 (default)		
HEX1[0]	Output	PIN_C18	7	B7_N0	PIN_C18	3.3-V LVTTL		8mA (default)	2 (default)		
HEX2[6]	Output	PIN_B22	6	B6_N0	PIN_B22	3.3-V LVTTL		8mA (default)	2 (default)		
HEX2[5]	Output	PIN_C22	6	B6_N0	PIN_C22	3.3-V LVTTL		8mA (default)	2 (default)		
HEX2[4]	Output	PIN_B21	6	B6_N0	PIN_B21	3.3-V LVTTL		8mA (default)	2 (default)		
HEX2[3]	Output	PIN_A21	6	B6_N0	PIN_A21	3.3-V LVTTL		8mA (default)	2 (default)		
HEX2[2]	Output	PIN_B19	7	B7_N0	PIN_B19	3.3-V LVTTL		8mA (default)	2 (default)		
HEX2[1]	Output	PIN_A20	7	B7_N0	PIN_A20	3.3-V LVTTL		8mA (default)	2 (default)		
HEX2[0]	Output	PIN_B20	6	B6_N0	PIN_B20	3.3-V LVTTL		8mA (default)	2 (default)		
HEX2[5](n)	Output					3.3-V LVTTL		minimum current	2 (default)		
HEX3[6]	Output	PIN_E17	6	B6_N0	PIN_E17	3.3-V LVTTL		8mA (default)	2 (default)		
HEX3[5]	Output	PIN_D19	6	B6_N0	PIN_D19	3.3-V LVTTL		8mA (default)	2 (default)		
HEX3[4]	Output	PIN_C20	6	B6_N0	PIN_C20	3.3-V LVTTL		8mA (default)	2 (default)		
HEX3[3]	Output	PIN_C19	7	B7_N0	PIN_C19	3.3-V LVTTL		8mA (default)	2 (default)		
HEX3[2]	Output	PIN_E21	6	B6_N0	PIN_E21	3.3-V LVTTL		8mA (default)	2 (default)		
HEX3[1]	Output	PIN_E22	6	B6_N0	PIN_E22	3.3-V LVTTL		8mA (default)	2 (default)		
HEX3[0]	Output	PIN_F21	6	B6_N0	PIN_F21	3.3-V LVTTL		8mA (default)	2 (default)		
Ouverture	Output	PIN_A8	7	B7_N0	PIN_A8	3.3-V LVTTL		8mA (default)	2 (default)		
rst_n	Input	PIN_B8	7	B7_N0	PIN_B8	3.3-V LVTTL		8mA (default)			
Buscode[4]	Unknown					3.3-V LVTTL		8mA (default)			
Buscode[5]	Unknown					3.3-V LVTTL		8mA (default)			
Buscode[15]	Unknown					3.3-V LVTTL		8mA (default)			
Buscode[13]	Unknown					3.3-V LVTTL		8mA (default)			
Buscode[14]	Unknown					3.3-V LVTTL		8mA (default)			
Buscode[12]	Unknown					3.3-V LVTTL		8mA (default)			
Buscode[11]	Unknown					3.3-V LVTTL		8mA (default)			
Buscode[10]	Unknown					3.3-V LVTTL		8mA (default)			
Buscode[9]	Unknown					3.3-V LVTTL		8mA (default)			
Buscode[8]	Unknown					3.3-V LVTTL		8mA (default)			
Buscode[7]	Unknown					3.3-V LVTTL		8mA (default)			
Buscode[6]	Unknown					3.3-V LVTTL		8mA (default)			
Buscode[3]	Unknown					3.3-V LVTTL		8mA (default)			
Buscode[2]	Unknown					3.3-V LVTTL		8mA (default)			
Buscode[1]	Unknown					3.3-V LVTTL		8mA (default)			
Buscode[0]	Unknown					3.3-V LVTTL		8mA (default)			
Buscode	Unknown					3.3-V LVTTL		8mA (default)			
<<new node>>											

Figure 12 : Assignment pin

Sources

Lors de ce projet différentes sources ont été utiliser pour ce mini-projet, voici les liens de ceux-ci.

https://irfu.cea.fr/en/Phocea/file.php?class=cours&file=/herve.le-provost/langageVHDL.pdf

https://rti.etf.bg.ac.rs/rti/ri5rvl/tutorial/Html/Tuttbch/Tuttbch.htm#:~:text=A%20test%20bench%20is%20a,ports%20of%20the%20UUT%20entity.

https://www.eng.auburn.edu/~nelson/courses/elec4200/Slides/VHDL%206%20Testbench.pdf

https://hal.science/hal-04301440v1/document

https://stackoverflow.com/questions/29267616/vhdl-if-statements-and-process-names

Tables des figures

Figure 1 : Carte DE10-Lite 6
Figure 2 : Simulation détecteur de front 8
Figure 3 : Simulation bascule D 10
Figure 4 : Simulation registre à changement parallèle 11
Figure 5 : Logigramme du stockage code 12
Figure 6 : Code extrait top module permettant le stockage du code 13
Figure 7 : Schéma de ce composant 14
Figure 8 : Simulation décodeur 7 segment 14
Figure 9 : Simulation comparateur 4 bits 15
Figure 10 : Simulation comparateur 16 bits 16
Figure 11 : Schéma Comparateur 16 bits 17
Figure 12 : Assignment pin 19